

OUTPUTS

```
Attempting to create 'data' DataFrame...
DataFrame 'data' creation attempt finished.
Successfully created 'data' DataFrame. Displaying head and info:
```

	Temp	Rainfall	Humidity	Soil_Moisture	Nitrogen	Disease	Yield
0	30	200	80	40	60	1	0
1	25	150	60	35	55	0	1
2	28	180	70	38	58	1	1
3	32	220	85	42	65	1	0
4	27	160	65	36	57	0	1

```
DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Temp            5 non-null     int64
1   Rainfall        5 non-null     int64
2   Humidity        5 non-null     int64
3   Soil_Moisture   5 non-null     int64
4   Nitrogen        5 non-null     int64
5   Disease         5 non-null     int64
6   Yield           5 non-null     int64
dtypes: int64(7)
memory usage: 412.0 bytes
```

	Temp	Rainfall	Humidity	Soil_Moisture	Nitrogen	Disease	Yield
0	30	200	80	40	60	1	0
1	25	150	60	35	55	0	1
2	28	180	70	38	58	1	1
3	32	220	85	42	65	1	0
4	27	160	65	36	57	0	1

```
DataFrame info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Temp            5 non-null     int64
1   Rainfall        5 non-null     int64
2   Humidity        5 non-null     int64
3   Soil_Moisture   5 non-null     int64
4   Nitrogen        5 non-null     int64
5   Disease         5 non-null     int64
6   Yield           5 non-null     int64
dtypes: int64(7)
memory usage: 412.0 bytes
```

```
Initial S: ['0', '0', '0', '0']
Initial G: [['?', '?', '?', '?']]
```

```
Processing Example 1: ['High', 'High', 'Heavy', 'Wet'] -> Yes
```

```
S: ['High', 'High', 'Heavy', 'Wet']
```

```
G: [['?', '?', '?', '?']]
```

```
Processing Example 2: ['High', 'High', 'Light', 'Wet'] -> Yes
```

```
S: ['High', 'High', '?', 'Wet']
```

```
G: [['?', '?', '?', '?']]
```

```
Processing Example 3: ['Medium', 'High', 'Heavy', 'Wet'] -> Yes
```

```
S: ['?', 'High', '?', 'Wet']
```

```
G: [['?', '?', '?', '?']]
```

```
Processing Example 4: ['Low', 'Normal', 'Light', 'Dry'] -> No
```

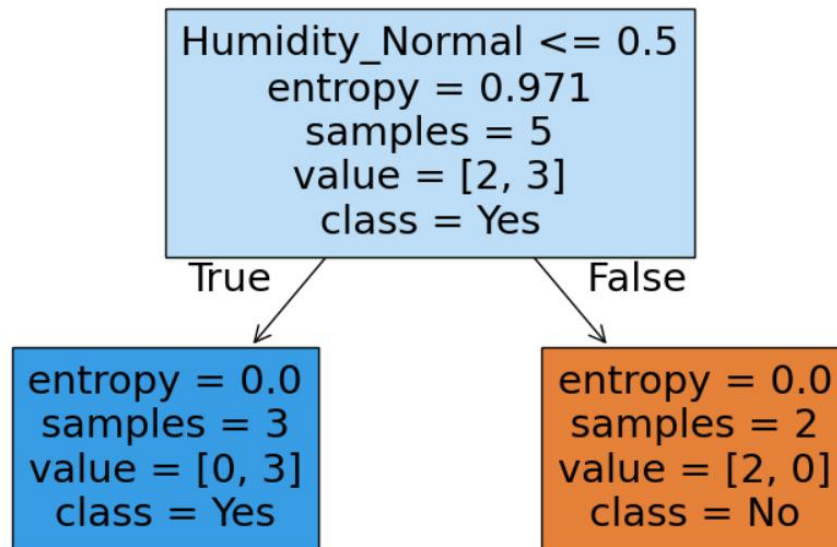
```
S: ['?', 'High', '?', 'Wet']
```

```
G: [['?', '?', '?', '?'], ['?', 'High', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', 'Wet']]
```

```
Processing Example 5: ['Medium', 'Normal', 'Light', 'Dry'] -> No
```

```
S: ['?', 'High', '?', 'Wet']
```

```
G: [['?', '?', '?', '?'], ['?', 'High', '?', '?'], ['?', '?', '?', '?'], ['?', '?', '?', 'Wet'], ['?',
```



```
from sklearn.naive_bayes import GaussianNB
```

```
model = GaussianNB()
model.fit(X, y)
```

```
pred = model.predict(X)
print(pred)
```

```
[1 1 1 0 0]
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
for k in range(1,6):
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X, y)
    print(f"K={k}, Accuracy={model.score(X,y)}")
```

```
|
```

```
K=1, Accuracy=1.0
K=2, Accuracy=1.0
K=3, Accuracy=1.0
K=4, Accuracy=1.0
K=5, Accuracy=0.6
```

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()
model.fit(X, y)
```

```
pred = model.predict(X)
print(pred)
```

```
[1.00000000e+00 1.00000000e+00 1.00000000e+00 2.22044605e-16
 4.44089210e-16]
```

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(5,5), max_iter=1000)
model.fit(X, y)

print("Accuracy:", model.score(X,y))
```

Accuracy: 1.0

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

pred = model.predict(X)

print("Accuracy:", accuracy_score(y, pred))
print("Precision:", precision_score(y, pred))
print("Recall:", recall_score(y, pred))
print("F1 Score:", f1_score(y, pred))
```

Accuracy: 1.0
Precision: 1.0
Recall: 1.0
F1 Score: 1.0